

1. Analyze and Preprocess data - Check if the dataset has missing values or has any other problem.
2. Feature Engineering
3. Determine the cluster number 3. a. Use elbow method as baseline 3. b. Apply other methods to compare with elbow
4. Use Pipeline 4. a. Use scaler and dimensional reduction (if it is necessary). 4. b. Use suitable clustering model for this problem.
5. Visualize your cluster in 2D or 3D to compare the result.
6. Measure performance of your clustering model.
7. Conclusion and Discussion (<= 200 words, also input your opinion or conclusion here)

How can I measure your point:

1. Your function is callable and runs correctly
2. The performance of your model (in full pipeline) is acceptable. The final error based on my train and test set is low enough.
3. The data preprocessing is correct or make sense
4. The Feature engineering is correct or make sense
5. Any other additional process will be considered a small plus point.

Biến	Mô tả
BALANCE	Số dư còn lại trong tài khoản để thực hiện mua sắm
BALANCE_FREQUENCY	Tần suất cập nhật số dư, giá trị từ 0 đến 1 (1 = cập nhật thường xuyên)
PURCHASES	Tổng số tiền đã mua sắm từ tài khoản
ONEOFF_PURCHASES	Giá trị lớn nhất của một lần mua hàng đơn lẻ
INSTALLMENTS_PURCHASES	Tổng giá trị mua hàng theo hình thức trả góp
CASH_ADVANCE	Số tiền mặt được ứng trước bởi người dùng
PURCHASES_FREQUENCY	Tần suất thực hiện các giao dịch mua sắm (0 đến 1)
ONEOFF_PURCHASES_FREQUENCY	Tần suất thực hiện các giao dịch mua đơn lẻ (0 đến 1)
PURCHASES_INSTALLMENTS_FREQUENCY	Tần suất mua hàng trả góp (0 đến 1)
CASH_ADVANCE_FREQUENCY	Tần suất ứng trước tiền mặt (0 đến 1)
CASH_ADVANCE_TRX	Số lượng giao dịch ứng trước tiền mặt
PURCHASES_TRX	Số lượng giao dịch mua hàng
CREDIT_LIMIT	Hạn mức tín dụng của thẻ đối với người dùng
PAYMENTS	Tổng số tiền người dùng đã thanh toán
MINIMUM_PAYMENTS	Khoản thanh toán tối thiểu người dùng đã trả
PRC_FULL_PAYMENT	Tỷ lệ phần trăm thanh toán toàn bộ nợ
TENURE	Thời gian sử dụng thẻ tín dụng của người dùng

Chỉ dùng các features trên để phân cụm

Load Dataset

In [288...

```
# from google.colab import drive
# drive.mount('/content/drive')
```

```
In [289... import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib import pyplot as plt
from sklearn.cluster import KMeans
from yellowbrick.cluster import KElbowVisualizer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

```
In [290... df = pd.read_csv("D:\\Introduction-to-Machine-Learning_MTH10353\\Homework\\5\\Clustered_Customer_Data.csv")
```

```
In [291... df.head()
```

Out[291...

	Unnamed: 0	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE
0	0	40.900749	0.818182	95.40	0.00	95.4	0.00
1	1	3202.467416	0.909091	0.00	0.00	0.0	6442.00
2	2	2495.148862	1.000000	773.17	773.17	0.0	0.00
3	3	1666.670542	0.636364	1499.00	1499.00	0.0	205.00
4	4	817.714335	1.000000	16.00	16.00	0.0	0.00

◀ ▶

Data Analysis

```
In [292... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8950 entries, 0 to 8949
Data columns (total 19 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Unnamed: 0                               8950 non-null   int64
1   BALANCE                                  8950 non-null   float64
2   BALANCE_FREQUENCY                       8950 non-null   float64
3   PURCHASES                               8950 non-null   float64
4   ONEOFF_PURCHASES                        8950 non-null   float64
5   INSTALLMENTS_PURCHASES                  8950 non-null   float64
6   CASH_ADVANCE                            8950 non-null   float64
7   PURCHASES_FREQUENCY                     8950 non-null   float64
8   ONEOFF_PURCHASES_FREQUENCY              8950 non-null   float64
9   PURCHASES_INSTALLMENTS_FREQUENCY        8950 non-null   float64
10  CASH_ADVANCE_FREQUENCY                   8950 non-null   float64
11  CASH_ADVANCE_TRX                        8950 non-null   int64
12  PURCHASES_TRX                           8950 non-null   int64
13  CREDIT_LIMIT                            8950 non-null   float64
14  PAYMENTS                                8950 non-null   float64
15  MINIMUM_PAYMENTS                        8950 non-null   float64
16  PRC_FULL_PAYMENT                        8950 non-null   float64
17  TENURE                                  8950 non-null   int64
18  Cluster                                 8950 non-null   int64
dtypes: float64(14), int64(5)
memory usage: 1.3 MB
```

```
In [293... df.describe()
```

Out[293...

	Unnamed: 0	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	
count	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000	8950.000000	
mean	4474.500000	1564.474828	0.877271	1003.204834	592.437371	411.067645	
std	2583.786788	2081.531879	0.236904	2136.634782	1659.887917	904.338115	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	2237.250000	128.281915	0.888889	39.635000	0.000000	0.000000	
50%	4474.500000	873.385231	1.000000	361.280000	38.000000	89.000000	
75%	6711.750000	2054.140036	1.000000	1110.130000	577.405000	468.637500	
max	8949.000000	19043.138560	1.000000	49039.570000	40761.250000	22500.000000	

In [294...

```
print("Missing values:")
df.isnull().sum()
```

Missing values:

Unnamed: 0	0
BALANCE	0
BALANCE_FREQUENCY	0
PURCHASES	0
ONEOFF_PURCHASES	0
INSTALLMENTS_PURCHASES	0
CASH_ADVANCE	0
PURCHASES_FREQUENCY	0
ONEOFF_PURCHASES_FREQUENCY	0
PURCHASES_INSTALLMENTS_FREQUENCY	0
CASH_ADVANCE_FREQUENCY	0
CASH_ADVANCE_TRX	0
PURCHASES_TRX	0
CREDIT_LIMIT	0
PAYMENTS	0
MINIMUM_PAYMENTS	0
PRC_FULL_PAYMENT	0
TENURE	0
Cluster	0
dtype: int64	

In [295...

```
df = df.drop(columns=["Unnamed: 0"])
df.head()
```

Out[295...

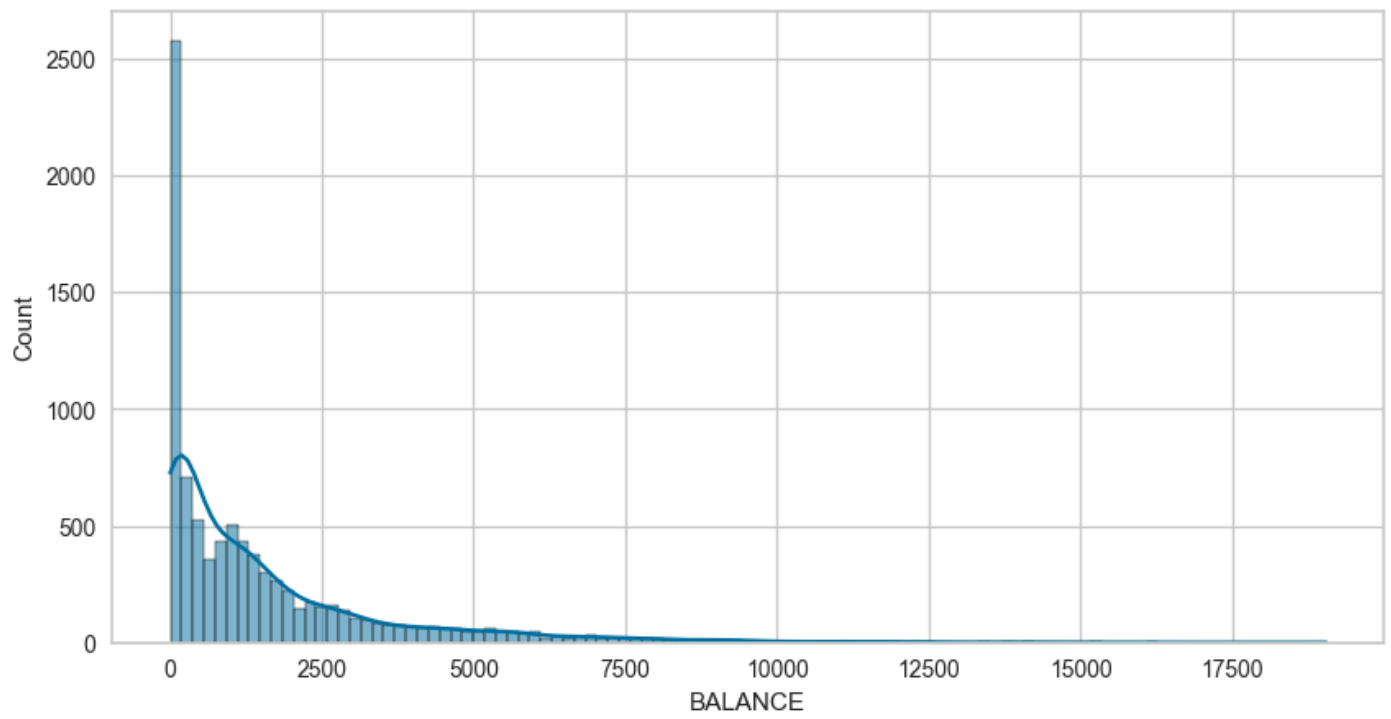
	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PU
0	40.900749	0.818182	95.40	0.00	95.4	0.000000	
1	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	
2	2495.148862	1.000000	773.17	773.17	0.0	0.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	
4	817.714335	1.000000	16.00	16.00	0.0	0.000000	

In [296...

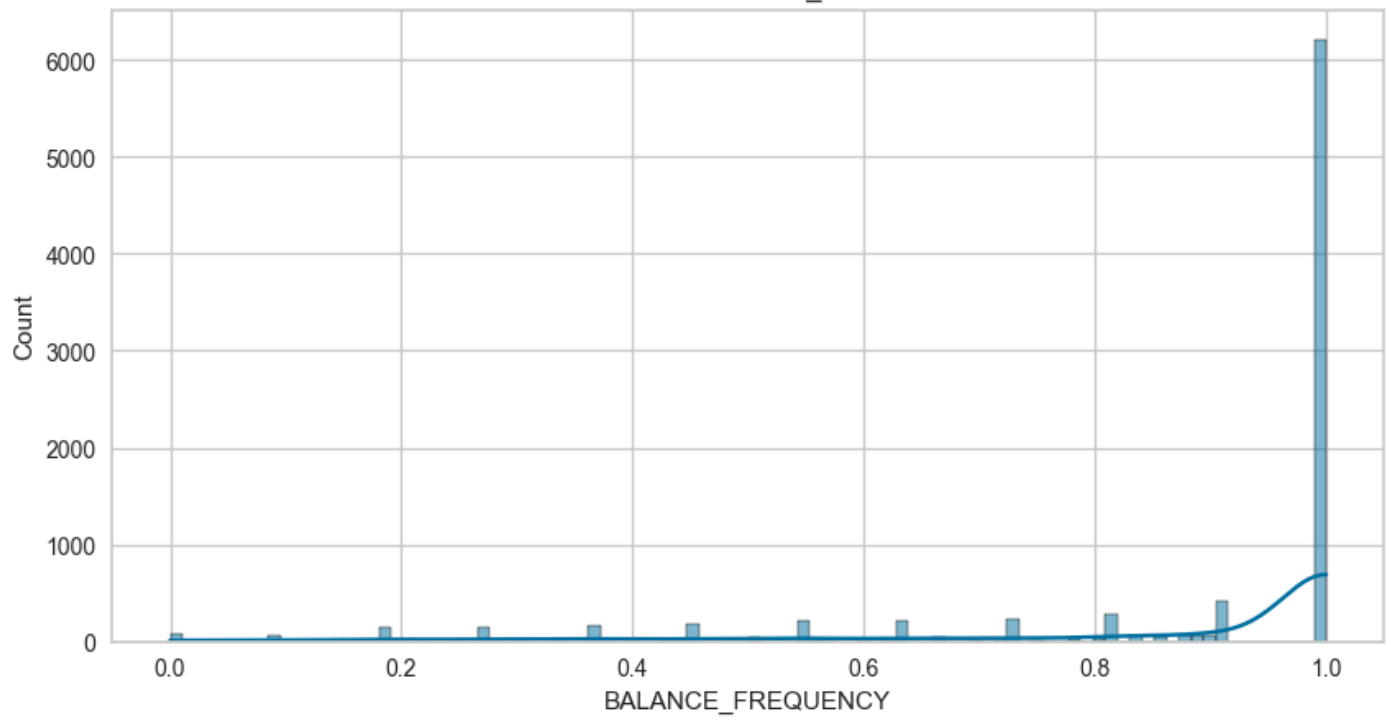
```
non_binary_columns = [col for col in df.columns if df[col].nunique() > 2]
for column in non_binary_columns:
    plt.figure(figsize=(10, 5))
    sns.histplot(df[column], kde=True)
    plt.title(f"Distribution of {column}")
    plt.show()

binary_columns = [col for col in df.columns if df[col].nunique() == 2]
for column in binary_columns:
    plt.figure(figsize=(6, 4))
    sns.countplot(x=df[column])
    plt.title(f'Count of {column.capitalize()}')
    plt.xlabel(column.capitalize())
    plt.ylabel('Count')
    plt.show()
```

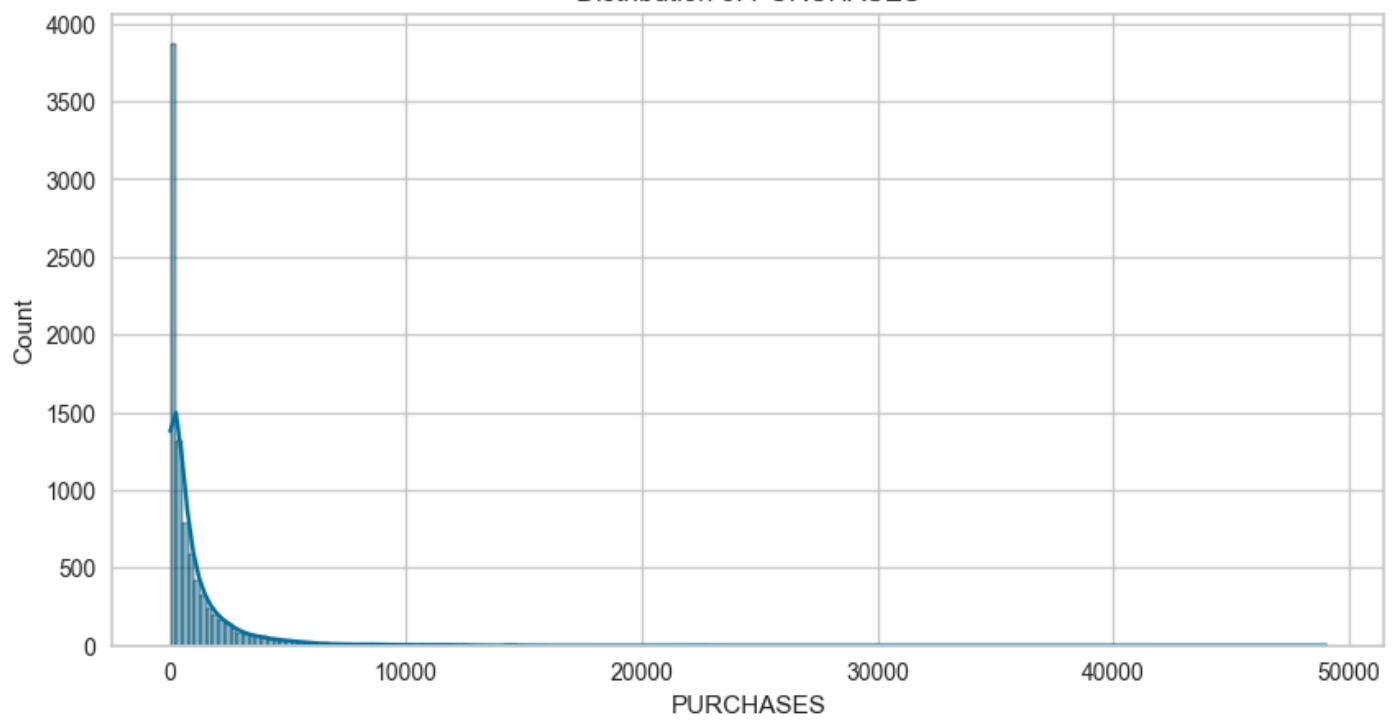
Distribution of BALANCE



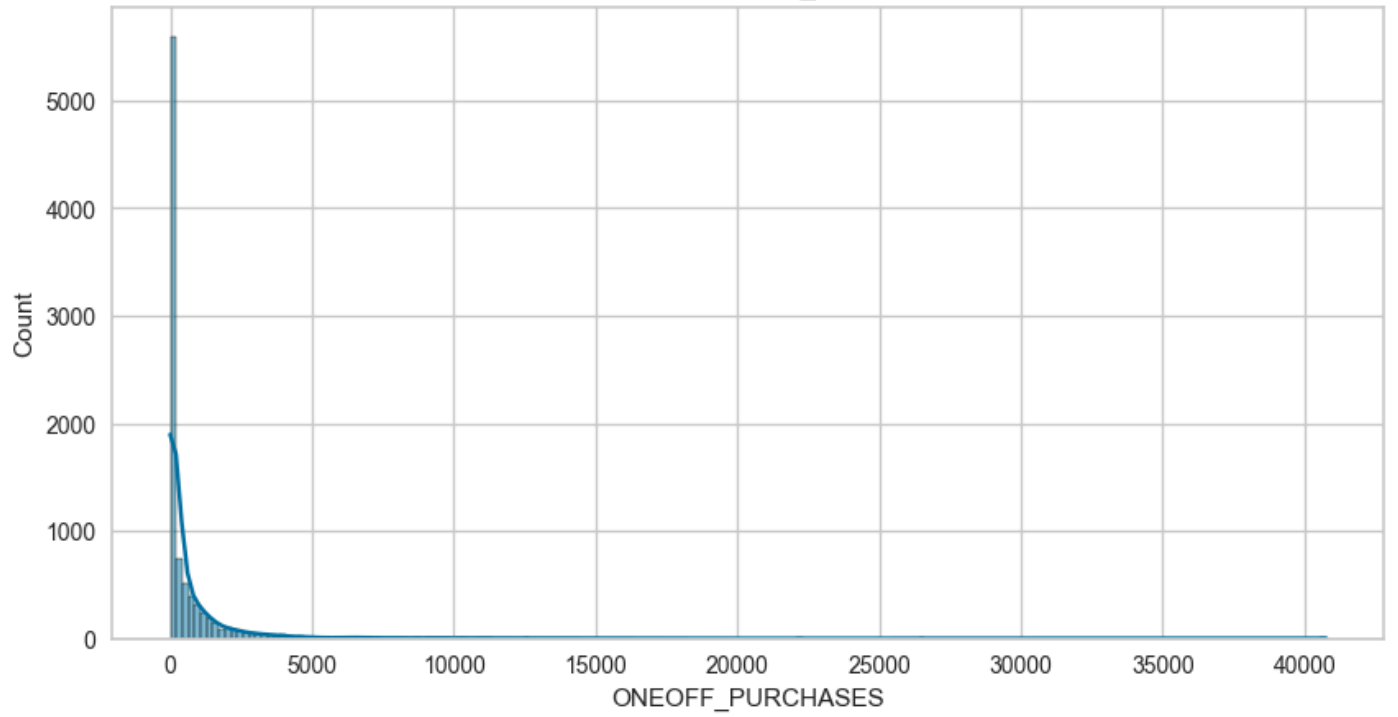
Distribution of BALANCE_FREQUENCY



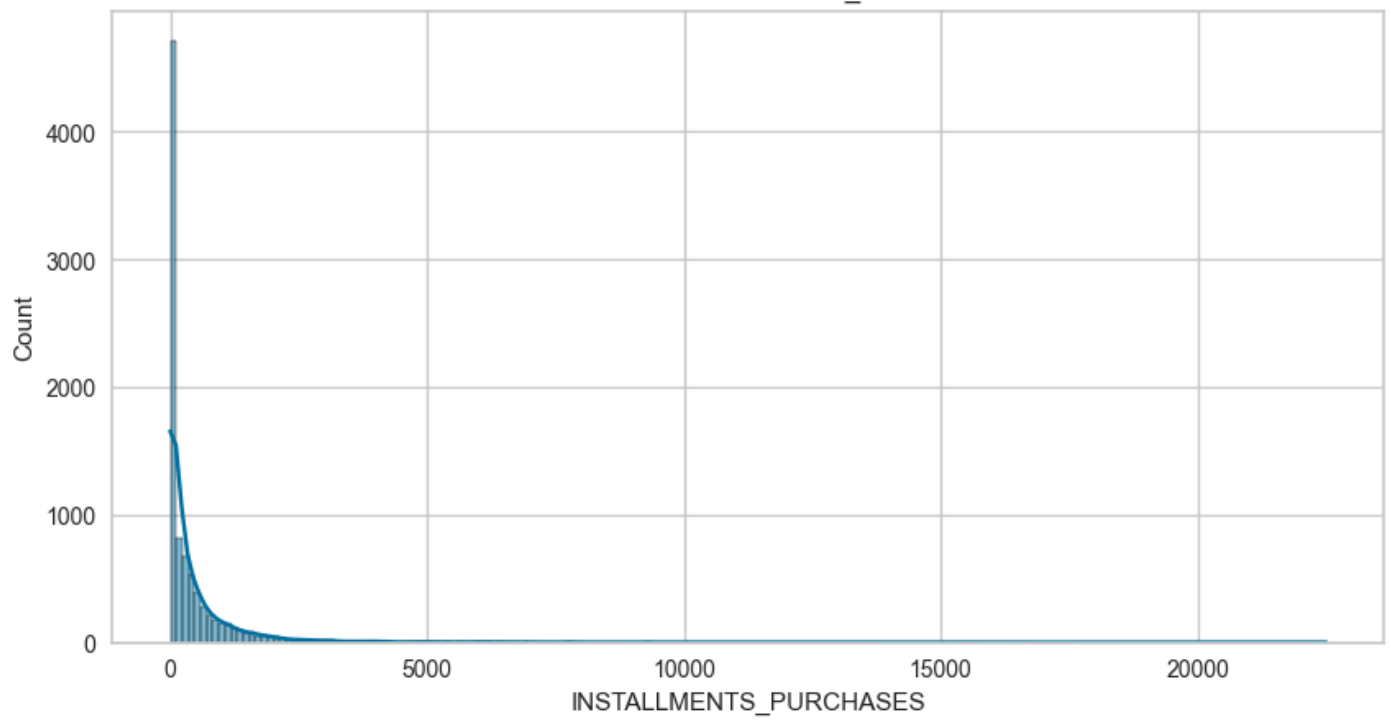
Distribution of PURCHASES



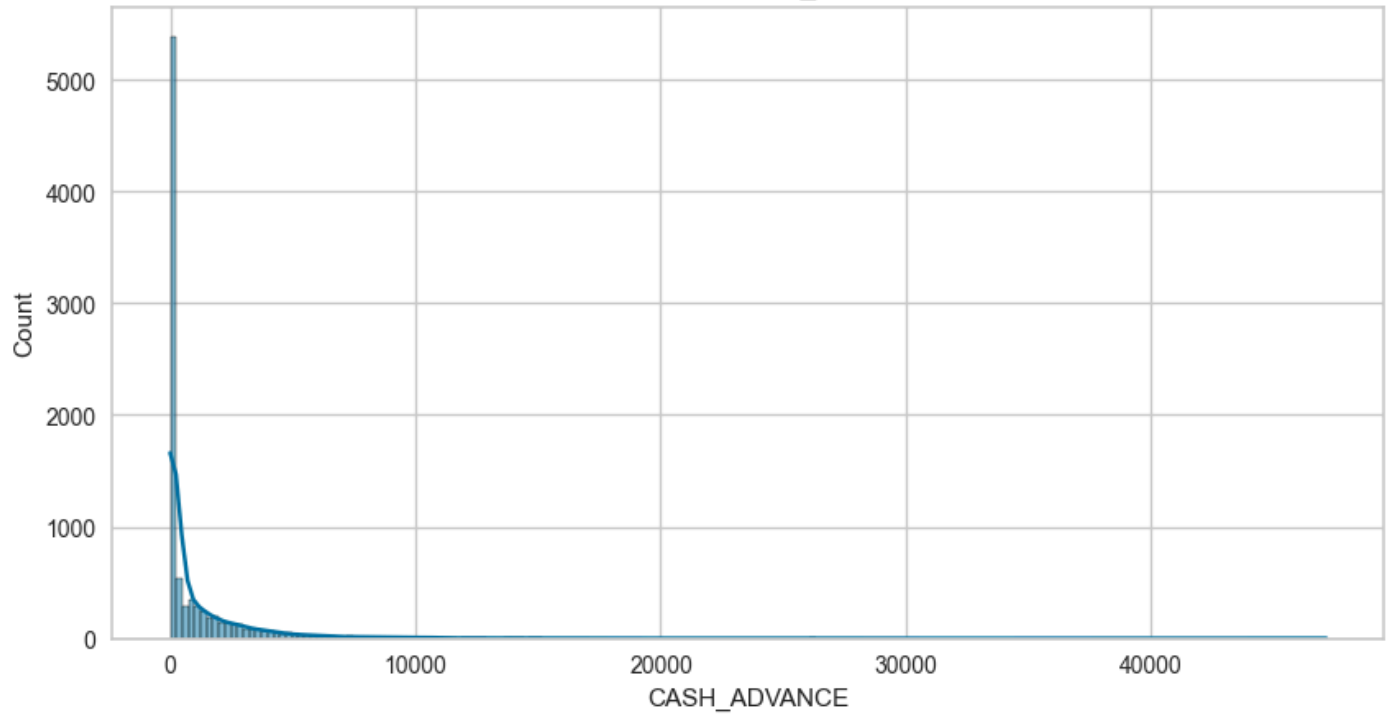
Distribution of ONEOFF_PURCHASES



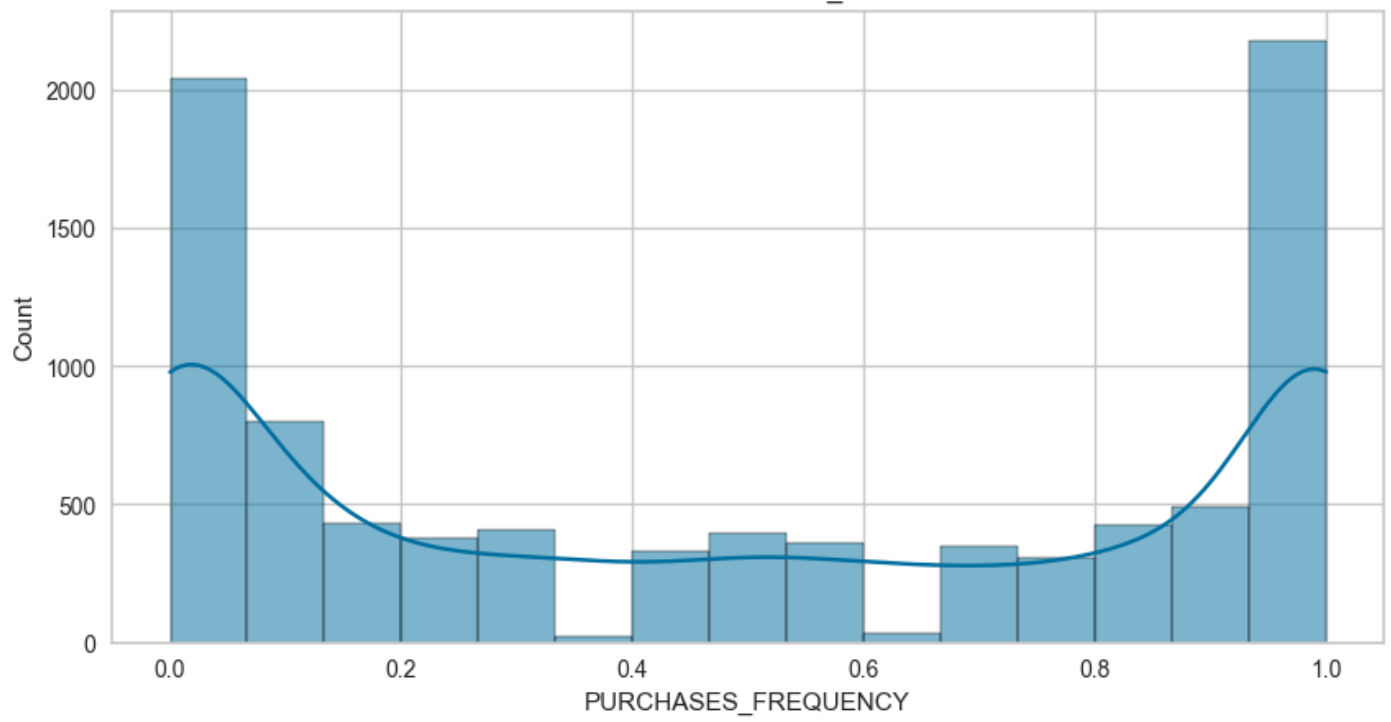
Distribution of INSTALLMENTS_PURCHASES



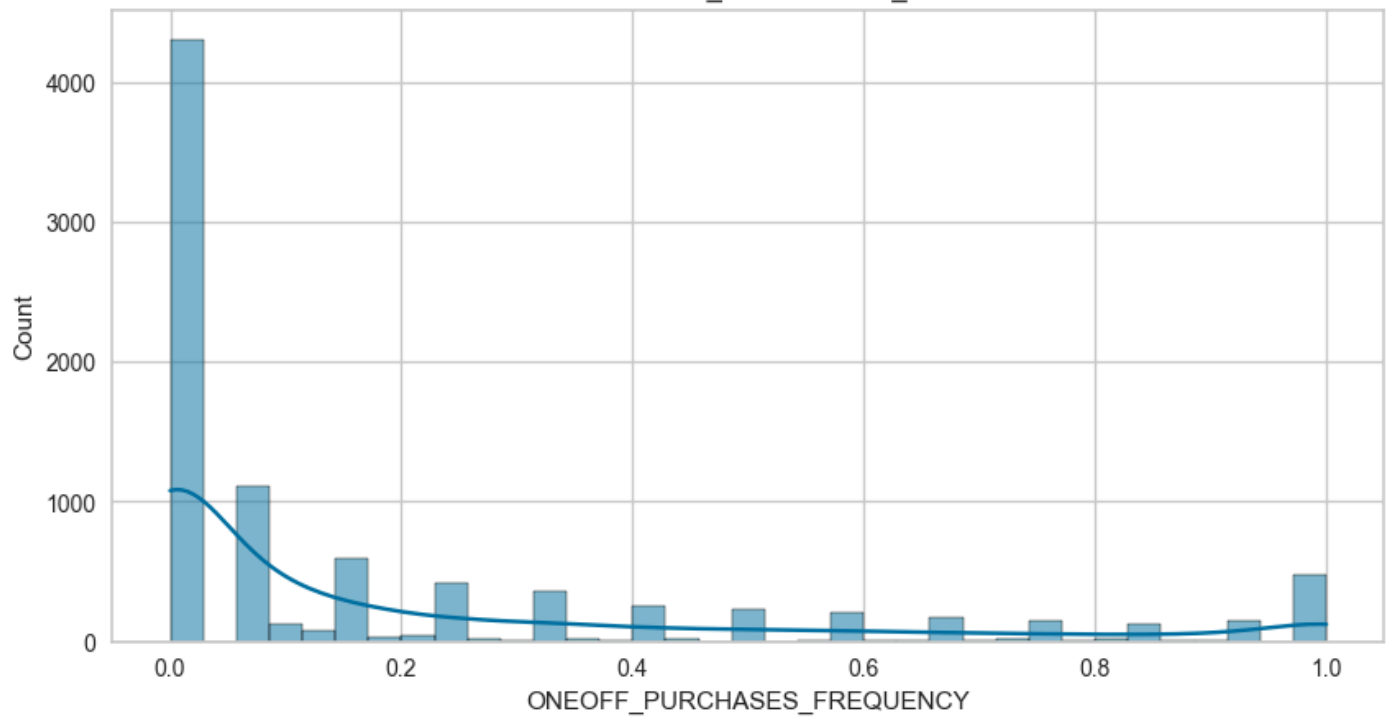
Distribution of CASH_ADVANCE



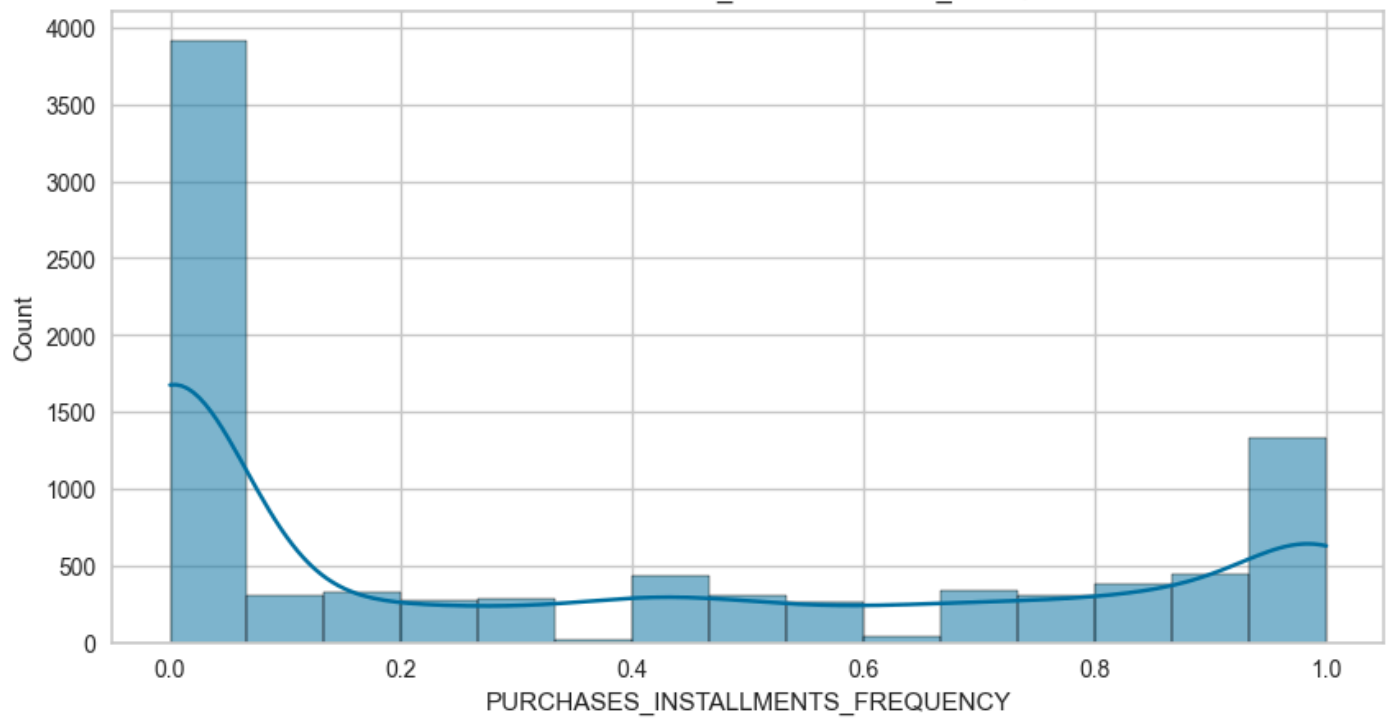
Distribution of PURCHASES_FREQUENCY



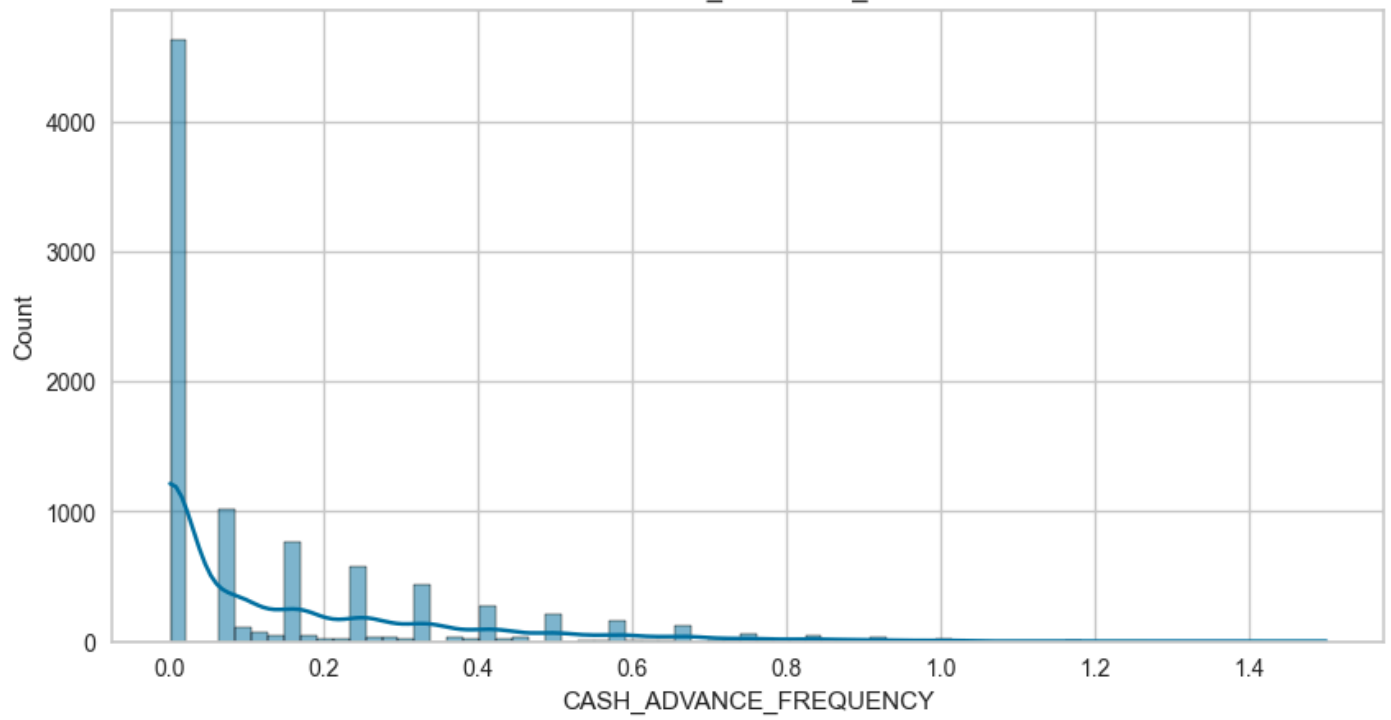
Distribution of ONEOFF_PURCHASES_FREQUENCY



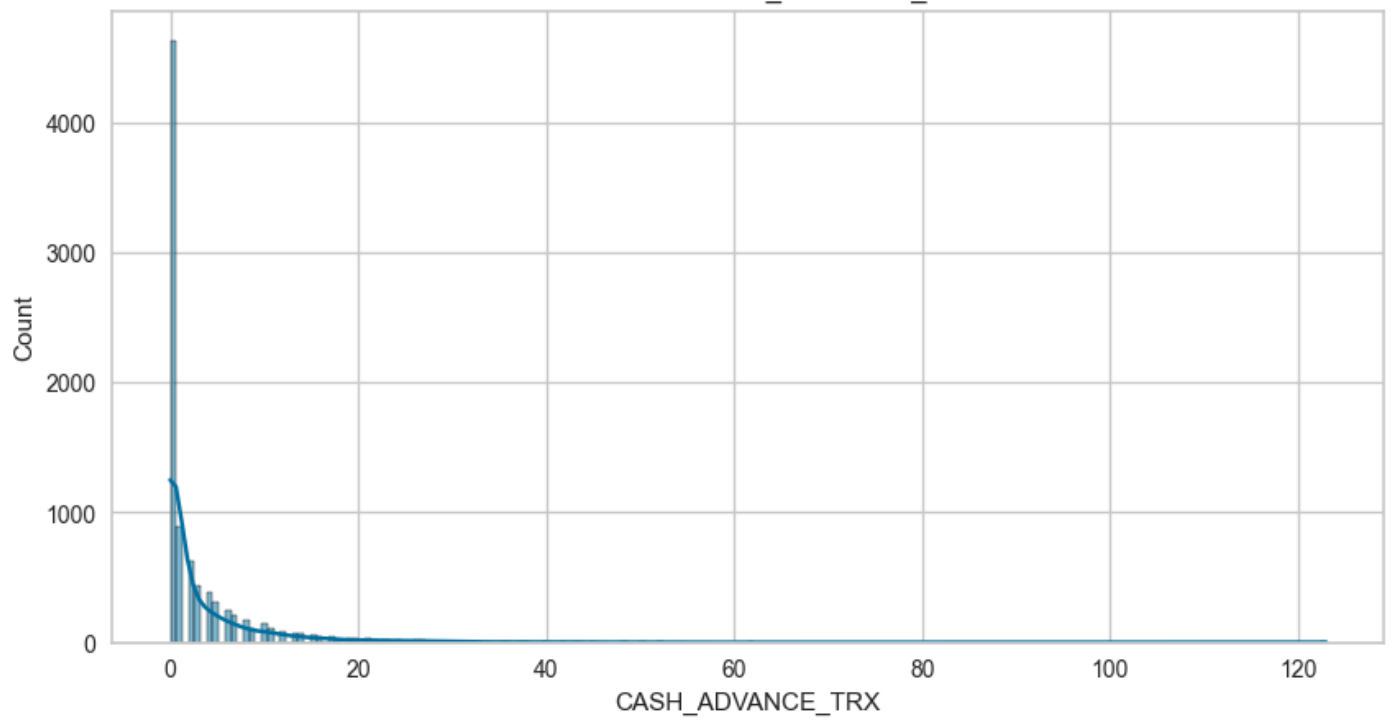
Distribution of PURCHASES_INSTALLMENTS_FREQUENCY



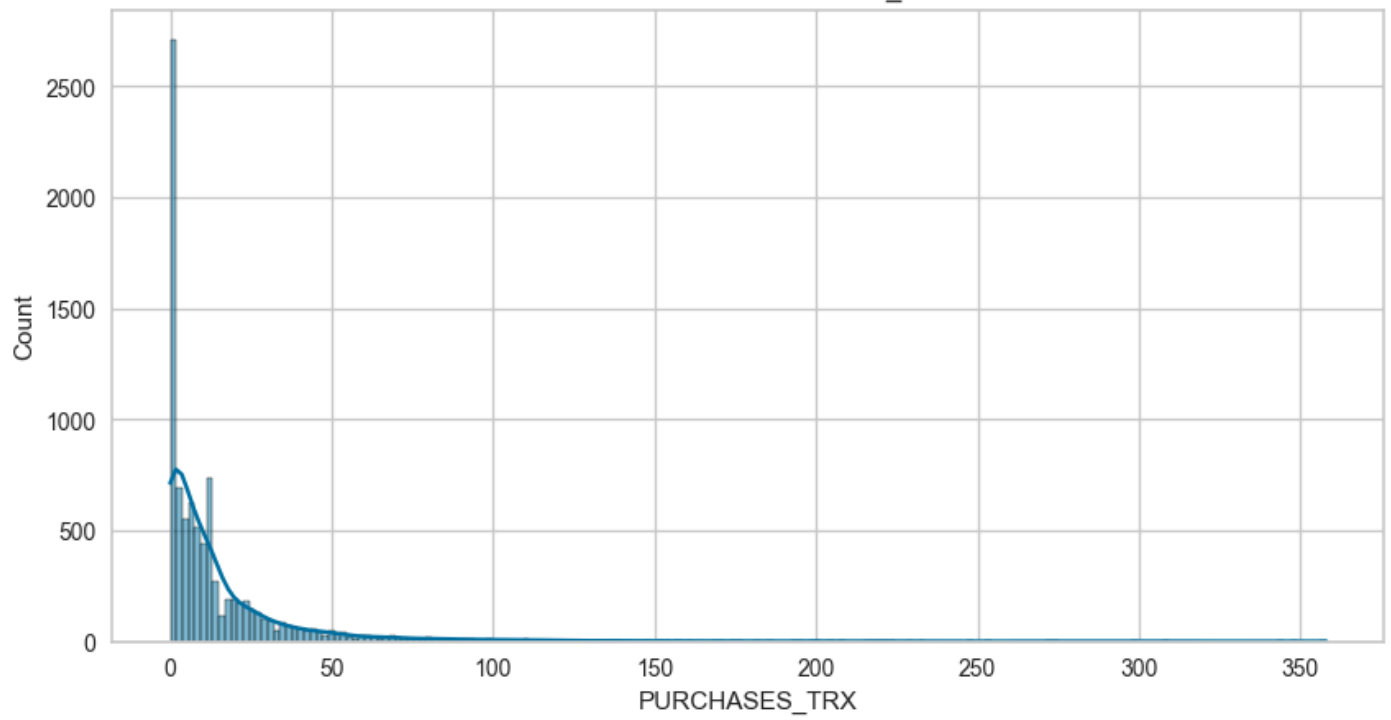
Distribution of CASH_ADVANCE_FREQUENCY



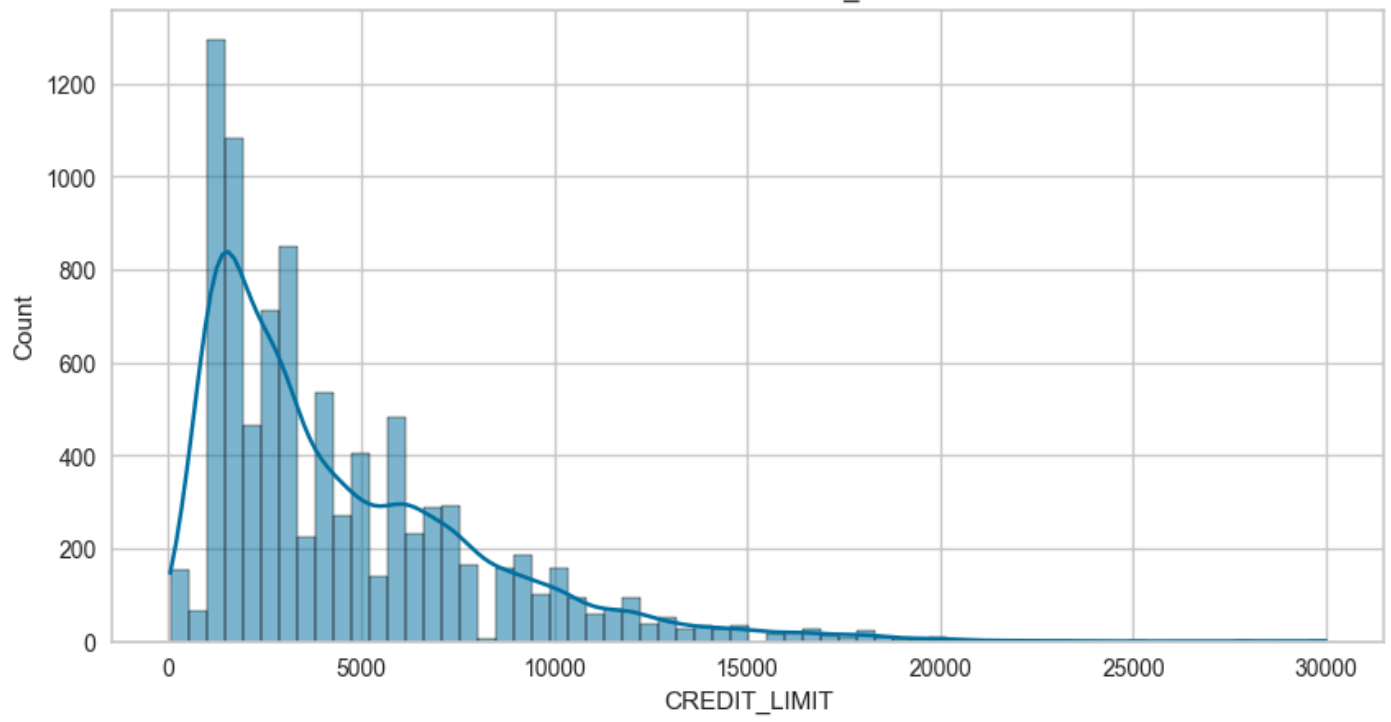
Distribution of CASH_ADVANCE_TRX



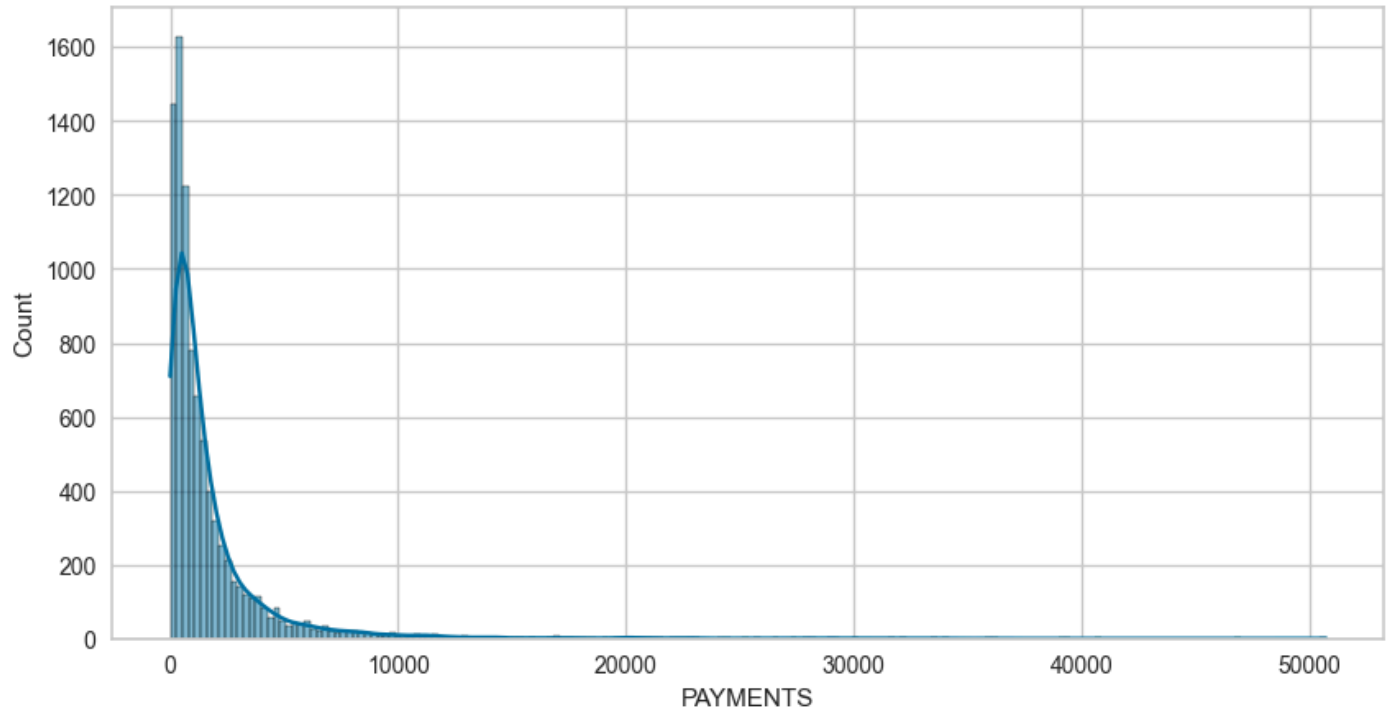
Distribution of PURCHASES_TRX



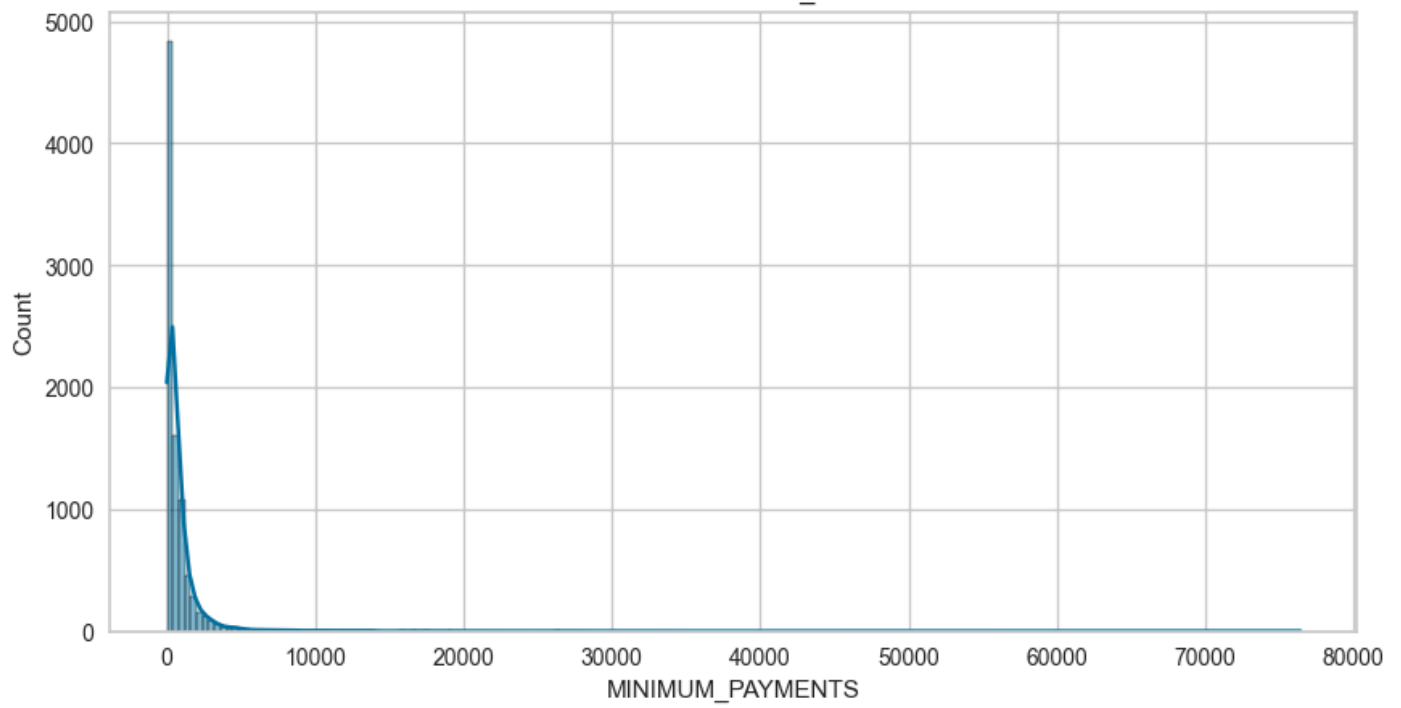
Distribution of CREDIT_LIMIT



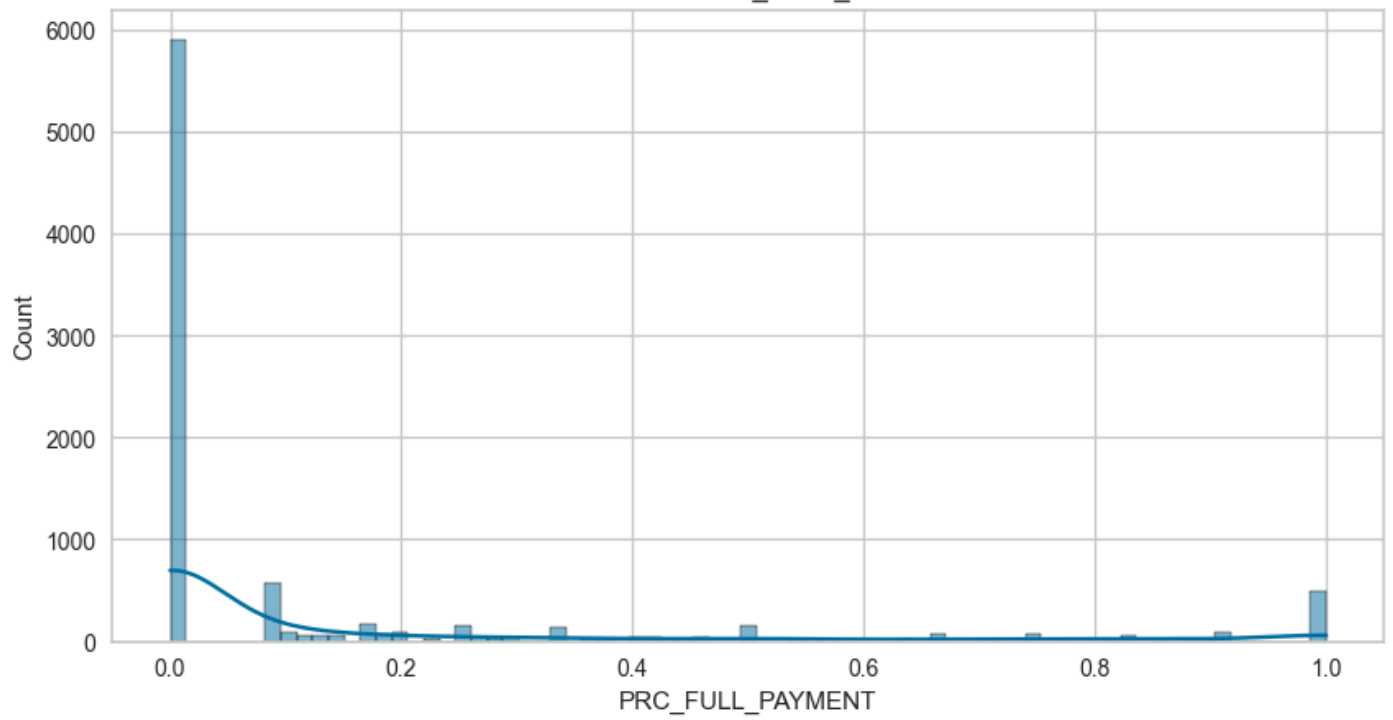
Distribution of PAYMENTS

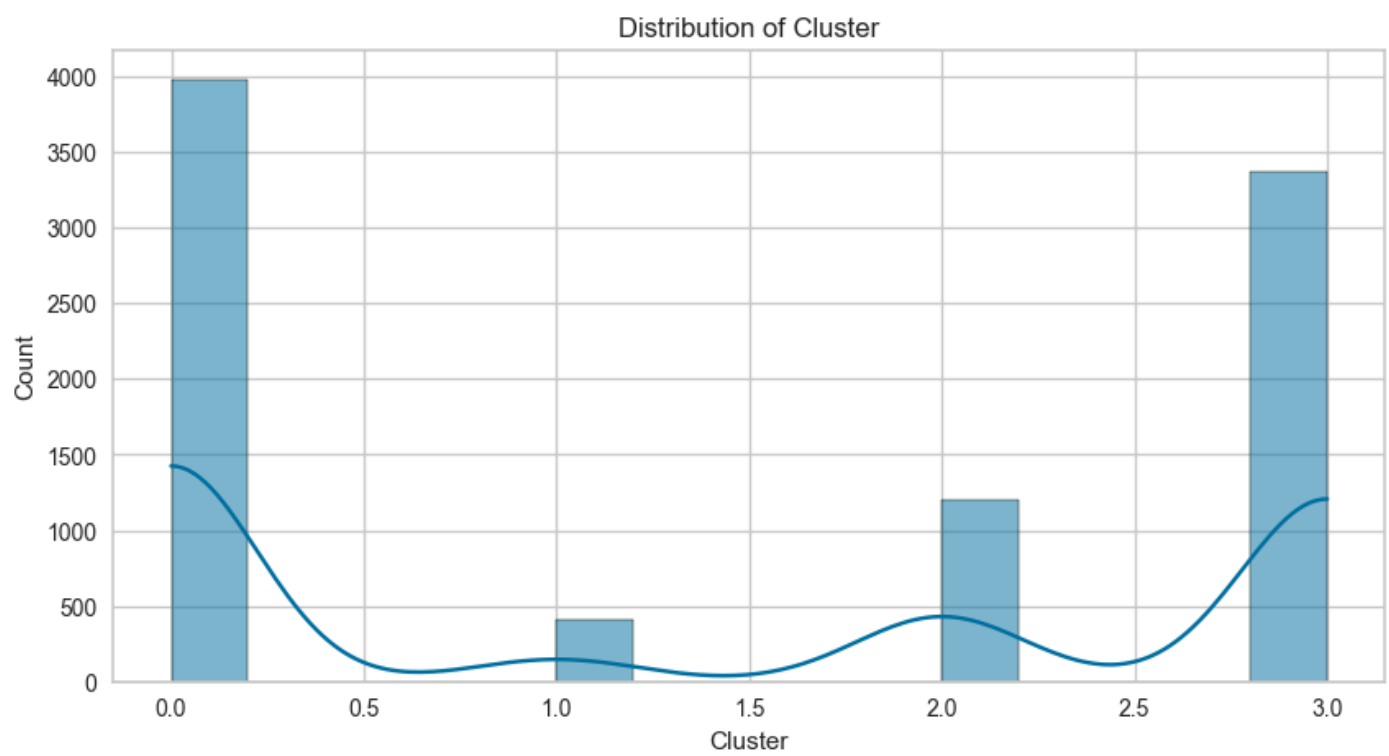
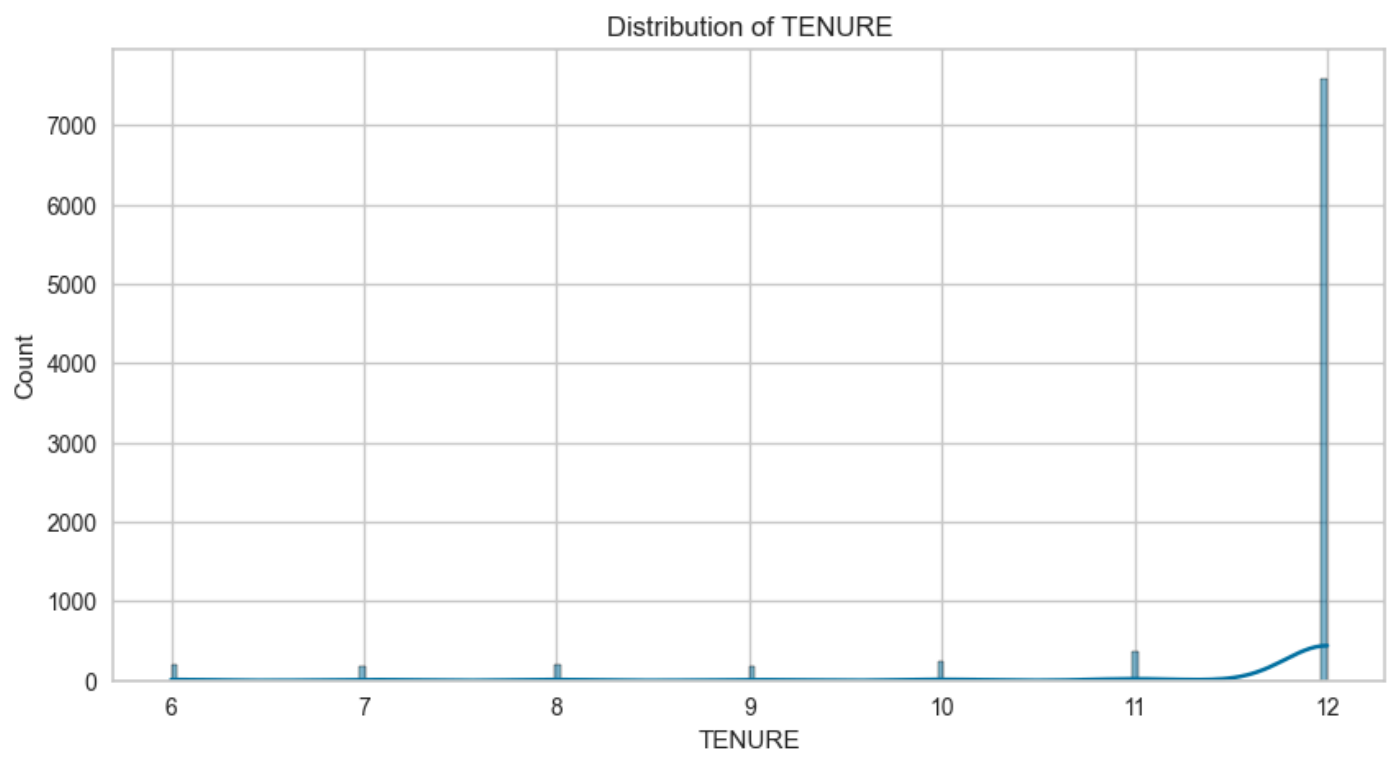


Distribution of MINIMUM_PAYMENTS

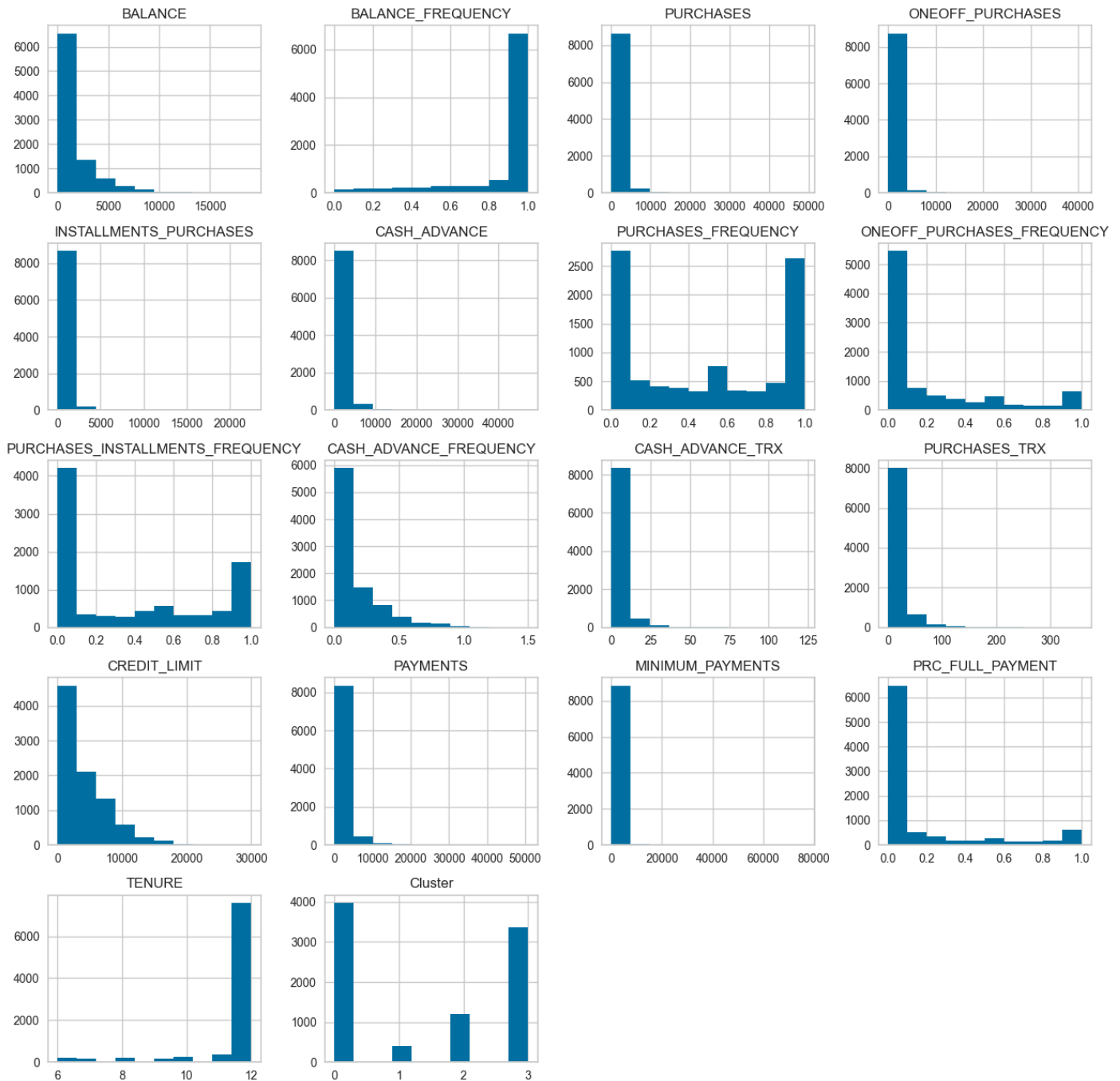


Distribution of PRC_FULL_PAYMENT





```
In [297... df.hist(figsize=(16, 16))  
plt.show()
```



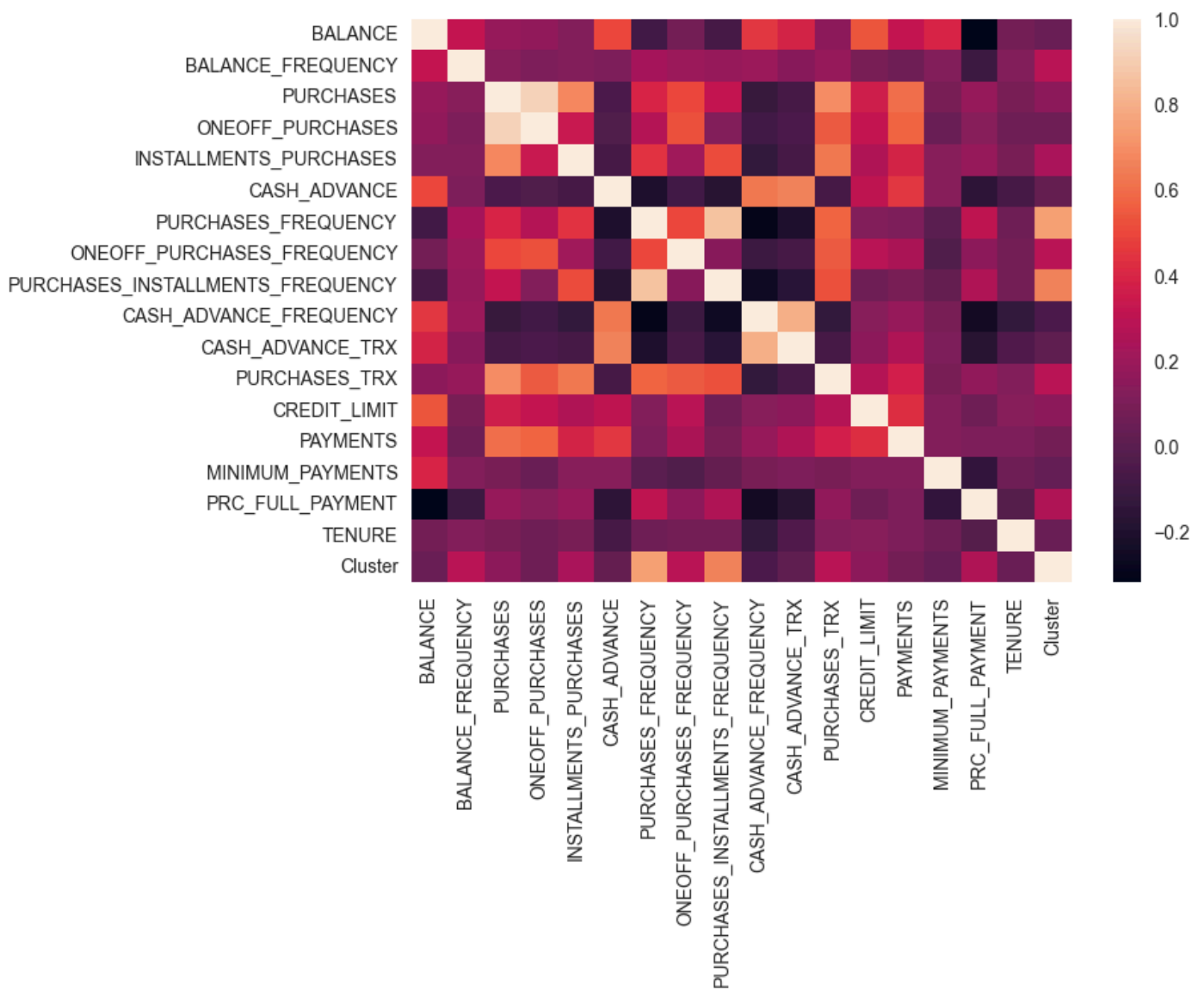
Data Preprocessing

```
In [298... def preprocessing_data(df):
    """
    Preprocess your data (eg. Drop null datapoints or fill missing data)
    :param df: pandas DataFrame
    :return: pandas DataFrame
    """
    # Todo: preprocess data
    df = df.dropna() # Drop rows with missing values
    return df
```

Feature Engineering

```
In [299... sns.heatmap(df.corr())
```

```
Out[299... <Axes: >
```

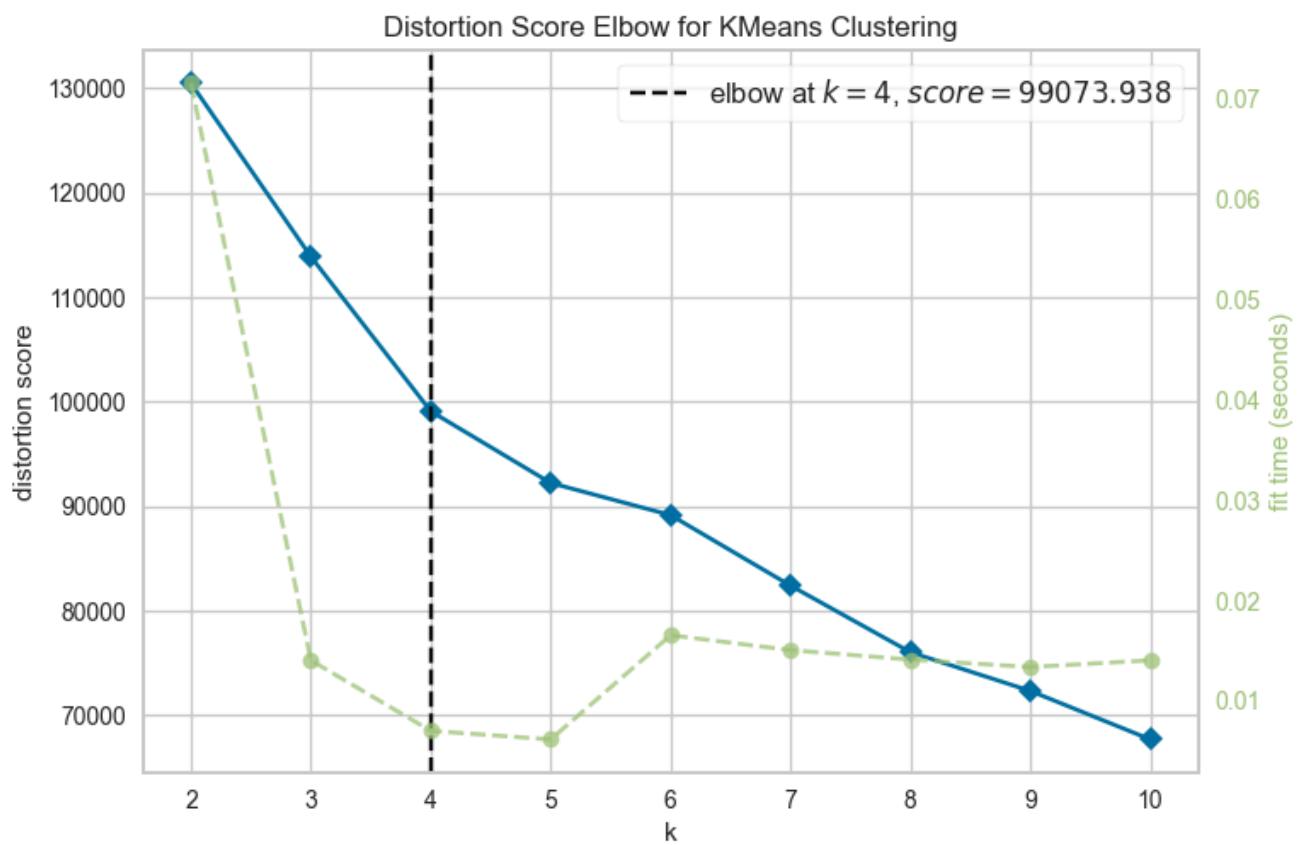


```
In [ ]: def apply_feature_engineering(df):
        scaler = StandardScaler()
        df = scaler.fit_transform(df)
        return df

df_scaled = apply_feature_engineering(df)
```

Model Part

```
In [319... elbow_method = KElbowVisualizer(KMeans(), k=(2,11), force_model=True)
elbow_method.fit(df_scaled)
elbow_method.show()
```



Out[319... <Axes: title={'center': 'Distortion Score Elbow for KMeans Clustering'}, xlabel='k', ylabel='distortion score'>

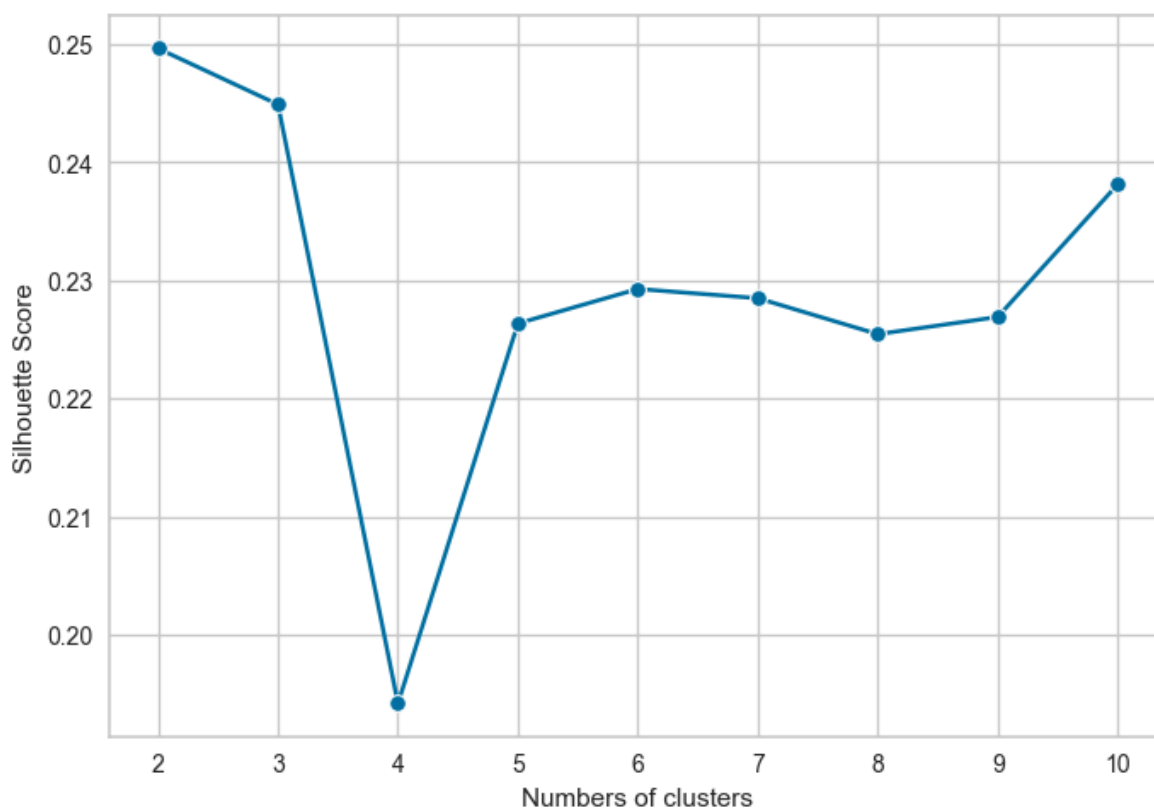
```
In [302... def kmeans_silhouette(num_clusters, x_vals):
    sil_score = []
    for num in num_clusters:
        kms = KMeans(n_clusters=num, random_state=42)
        kms.fit(x_vals)
        sil_score.append(silhouette_score(x_vals, kms.labels_))

    return sil_score

num_clusters = [i for i in range(2,11)]
sil_score = kmeans_silhouette(num_clusters, df_scaled)
sil_score
```

Out[302... [0.24965044892806315,
0.24490731625560425,
0.1941602879256082,
0.22635636203623352,
0.2292777287882891,
0.22849590130745884,
0.22545101072943752,
0.22690487131287546,
0.2381338621335736]

```
In [303... # Create a Line plot.
plot = sns.lineplot(x=num_clusters, y=sil_score, marker = 'o')
plot.set_xlabel("Numbers of clusters");
plot.set_ylabel("Silhouette Score");
```



```
In [304... kmeans_model = KMeans(n_clusters=4)
kmeans_model.fit(df_scaled)
```

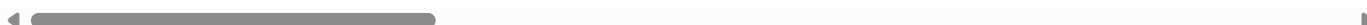
```
Out[304... KMeans ⓘ ?
Parameters
```

```
In [305... data_kmeans = df.copy()
data_kmeans['cluster_by_kmeans'] = kmeans_model.labels_
data_kmeans
```

```
Out[305...
```

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE
0	40.900749	0.818182	95.40	0.00	95.40	0.000000
1	3202.467416	0.909091	0.00	0.00	0.00	6442.945483
2	2495.148862	1.000000	773.17	773.17	0.00	0.000000
3	1666.670542	0.636364	1499.00	1499.00	0.00	205.788017
4	817.714335	1.000000	16.00	16.00	0.00	0.000000
...	...	...	...	...	...	...
8945	28.493517	1.000000	291.12	0.00	291.12	0.000000
8946	19.183215	1.000000	300.00	0.00	300.00	0.000000
8947	23.398673	0.833333	144.40	0.00	144.40	0.000000
8948	13.457564	0.833333	0.00	0.00	0.00	36.558778
8949	372.708075	0.666667	1093.25	1093.25	0.00	127.040008

8950 rows × 19 columns



Visualize your cluster in 2D or 3D to compare the result.

```
In [ ]: pca = PCA(n_components=len(df.columns))
pca.fit(df_scaled)
```


Out[]:

PCA

Parameters

```
In [308... pca = PCA(n_components=3) #dùng pca 3 thành phần
pca_df = pd.DataFrame(pca.fit_transform(df_scaled), columns=["pca_1", "pca_2", "pca_3"])#tạo ra dataframe với 3 t
```

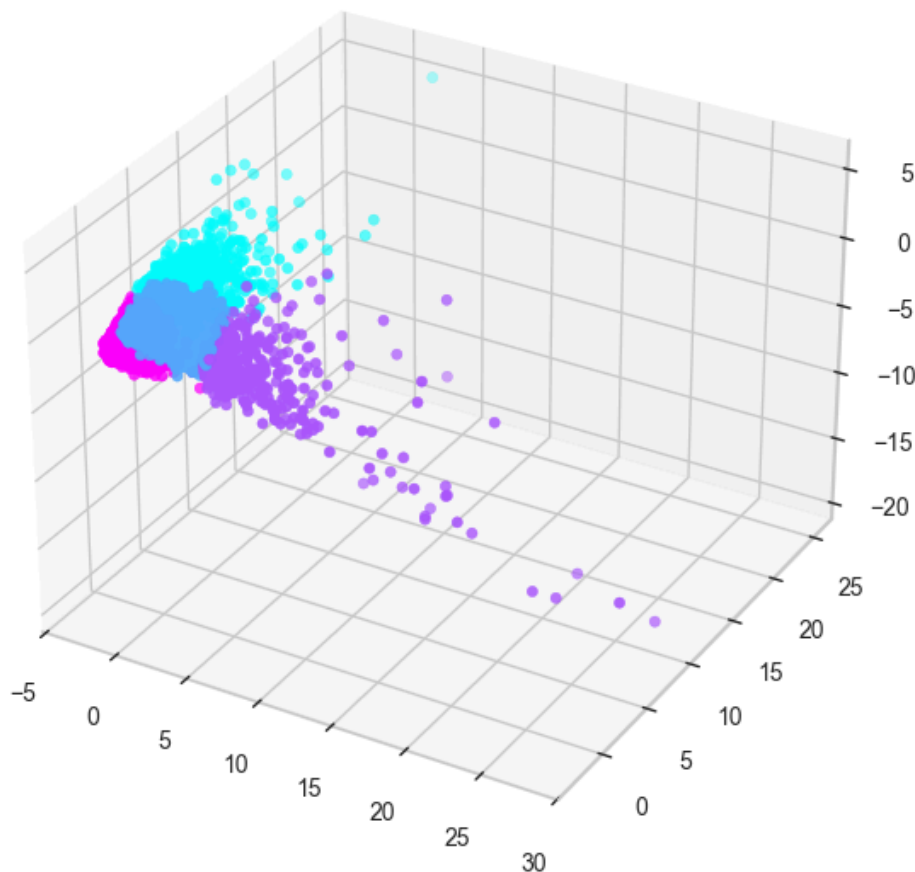
In [309... pca_df

Out[309...

	pca_1	pca_2	pca_3
0	-1.889102	-1.068519	-0.946078
1	-1.107088	2.412877	0.185523
2	1.230099	-0.409769	0.287999
3	-1.185467	0.111200	-1.809081
4	-1.821458	-0.678388	-0.891335
...	...	...	...
8945	0.086680	-2.151573	1.870919
8946	-0.134917	-1.782870	1.898902
8947	-0.505348	-1.960412	1.417109
8948	-2.525390	-0.693275	-0.737838
8949	-0.795960	-0.345823	-0.917649

8950 rows × 3 columns

```
In [310... plt.figure(figsize=(12, 7))
ax = plt.subplot(projection="3d")
ax.scatter(pca_df.pca_1, pca_df.pca_2, pca_df.pca_3, c=kmeans_model.labels_, cmap="cool")
plt.show()
```



```
In [311... pca = PCA(n_components=2)
pca.fit(df_scaled)
```

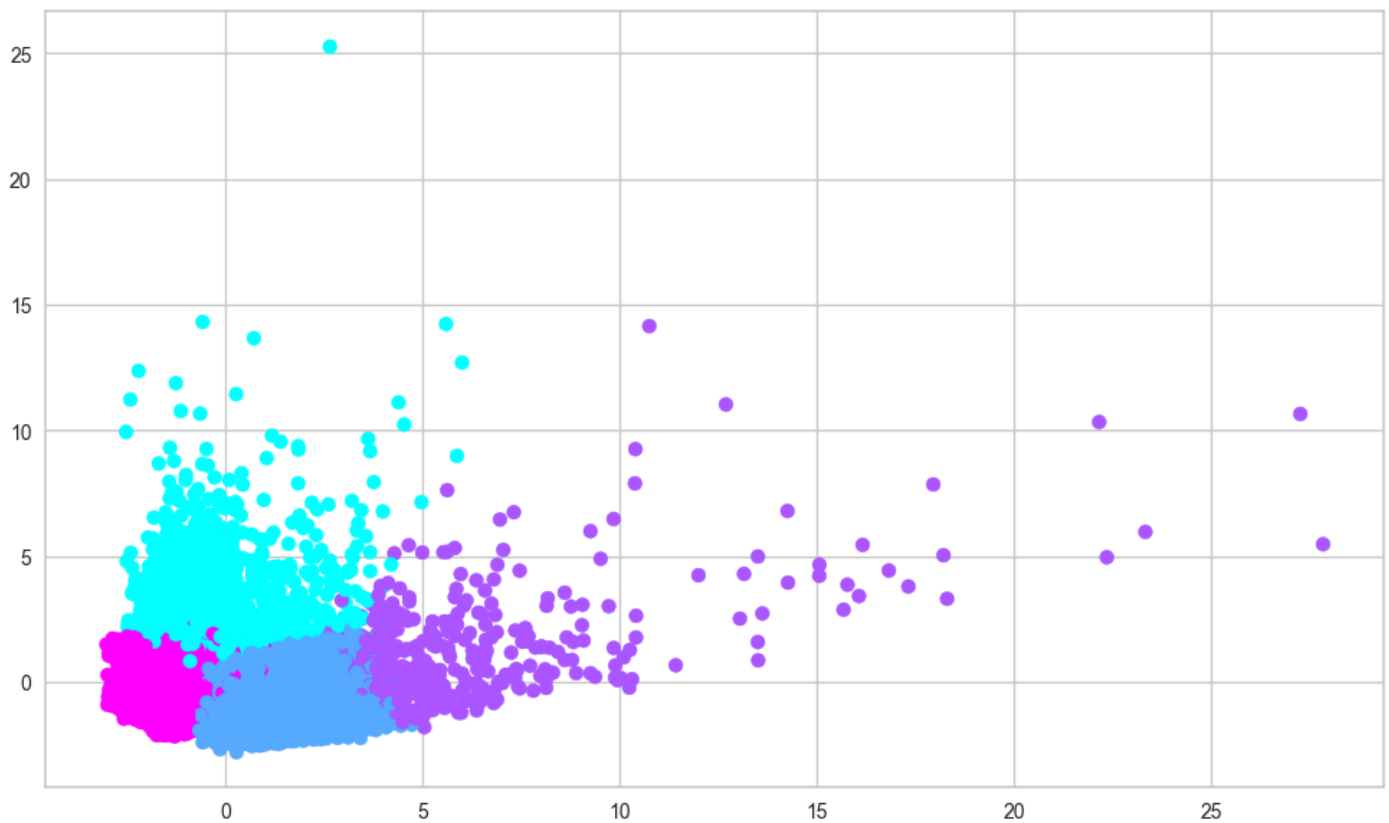
```
pca_df = pd.DataFrame(pca.transform(df_scaled), columns=["pca1", "pca2"])
```

In [318... `pca_df`

```
Out[318...
      pca1  pca2
0 -1.889102 -1.068519
1 -1.107088  2.412877
2  1.230099 -0.409769
3 -1.185467  0.111200
4 -1.821458 -0.678388
...      ...    ...
8945  0.086680 -2.151573
8946 -0.134917 -1.782870
8947 -0.505348 -1.960412
8948 -2.525390 -0.693275
8949 -0.795960 -0.345823
```

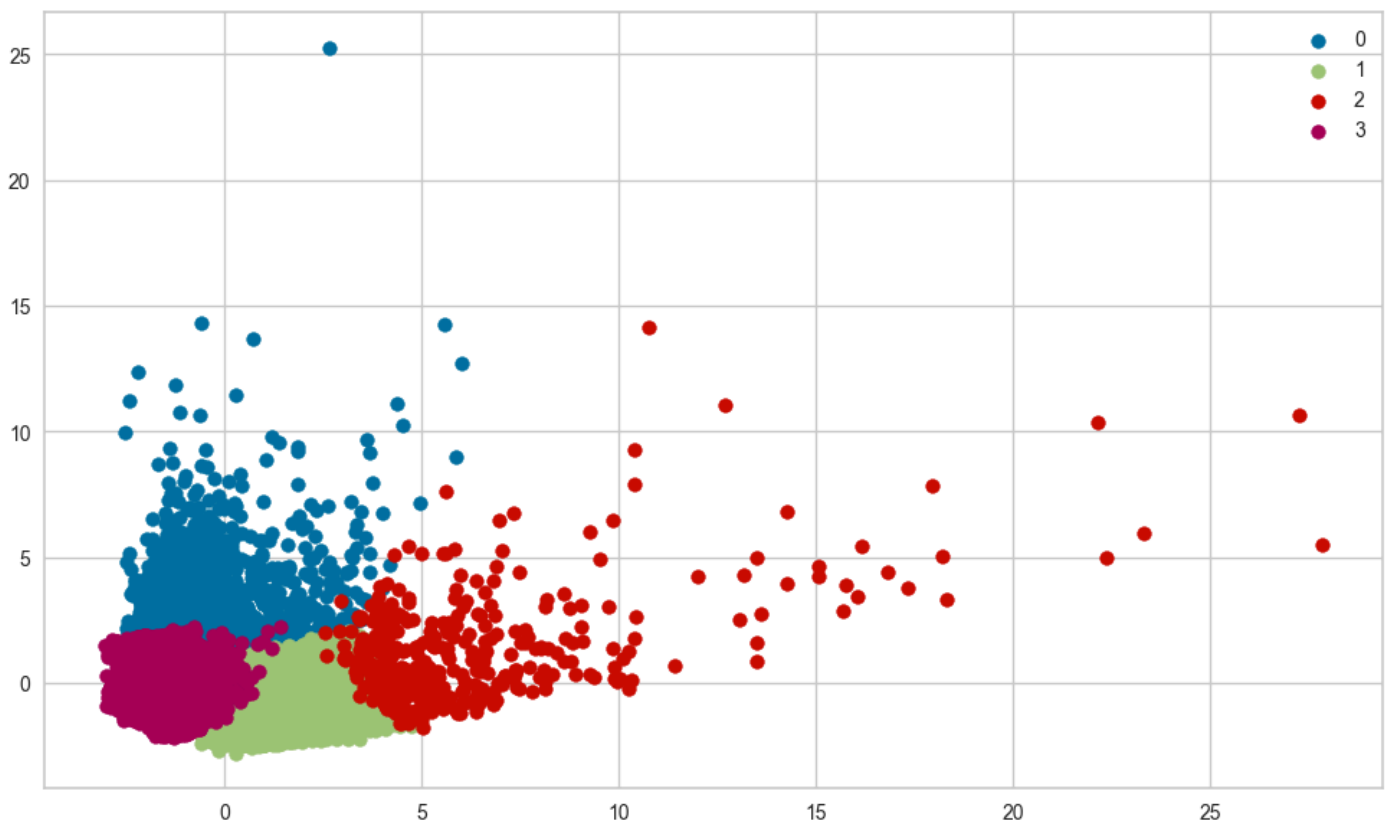
8950 rows × 2 columns

```
In [313...
plt.figure(figsize=(12, 7))
ax = plt.subplot()
ax.scatter(pca_df.pca1, pca_df.pca2, c=kmeans_model.labels_, cmap="cool")
plt.show()
```



```
In [317...
plt.figure(figsize=(12, 7))
ax = plt.subplot()
group = kmeans_model.labels_
for g in np.unique(group):
    index = np.where(group==g)
    ax.scatter(pca_df.iloc[index].pca1, pca_df.iloc[index].pca2, label=g, cmap="cool")
ax.legend()
plt.show()
```

C:\Users\ADMIN\AppData\Local\Temp\ipykernel_6264\3769084416.py:6: UserWarning: No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
ax.scatter(pca_df.iloc[index].pca1, pca_df.iloc[index].pca2, label=g, cmap="cool")



```
In [ ]: cluster = kmeans_model.labels_
cluster_0 = np.where(cluster==0) #index các data samples thuộc cluster 1
cluster_1 = np.where(cluster==1) #index các data samples thuộc cluster 2
cluster_2 = np.where(cluster==2) #index các data samples thuộc cluster 3
cluster_3 = np.where(cluster==3) #index các data samples thuộc cluster 4
```

```
In [320... #Lấy ra các data samples thuộc cluster 1
df.iloc[cluster_0]
#Lấy ra các data samples thuộc cluster 2
df.iloc[cluster_1]
#Lấy ra các data samples thuộc cluster 3
df.iloc[cluster_2]
#Lấy ra các data samples thuộc cluster 4
df.iloc[cluster_3]
print('Original distinct cluster labels present in cluster 0:\n',df.iloc[cluster_0]["Cluster"].nunique())
print('Original distinct cluster labels present in cluster 1:\n',df.iloc[cluster_1]["Cluster"].nunique())
print('Original distinct cluster labels present in cluster 2:\n',df.iloc[cluster_2]["Cluster"].nunique())
print('Original distinct cluster labels present in cluster 3:\n',df.iloc[cluster_3]["Cluster"].nunique())
```

Original distinct cluster labels present in cluster 0:

1

Original distinct cluster labels present in cluster 1:

1

Original distinct cluster labels present in cluster 2:

1

Original distinct cluster labels present in cluster 3:

1

Discussion & Conclusion